

Topic 1 - Comparison Operators

Explanation of Elements

Instead of finding an exact match, we often need to find a range of data. We do this using comparison operators:

\$gt

Greater Than (>)

\$lt

Less Than (<)

\$gte

Greater Than or Equal To (>=)

\$lte

Less Than or Equal To (<=)

Setup Data (Run this first!)

```
JavaScript
db.products.insertMany([
  { name: "Basic Mouse", category: "accessory", price: 25 },
  { name: "Mechanical Keyboard", category: "accessory", price: 120 },
  { name: "Budget Laptop", category: "laptop", price: 350 },
  { name: "Mid-Range Laptop", category: "laptop", price: 800 },
  { name: "Gaming Laptop", category: "laptop", price: 1500 }
])
```

Instructor Scenario: The Budget Filter

Scenario: A user is browsing the store and wants a mid-range laptop. They set the price filter to show items more expensive than \$400, but less than or equal to \$1000.

The Query:

```
JavaScript
db.products.find({
  category: "laptop",
  price: { $gt: 400, $lte: 1000 }
})
```

Student Exercise

☐ **Scenario:** A customer is on a strict budget. Write a query to find ALL products (regardless of category) that cost strictly less than \$150.

Topic 2 - Inclusion/Exclusion Operators

Explanation of Elements

Instead of writing multiple "or" conditions for the exact same field, you can use arrays:

\$in

Returns a document if the field matches any value inside the provided array.

\$nin

Returns a document only if the field matches none of the values inside the provided array.

Setup Data (Run this first!)

```
JavaScript
db.phones.insertMany([
  { model: "iPhone 14", brand: "Apple", stock: 50 },
  { model: "Galaxy S23", brand: "Samsung", stock: 30 },
  { model: "Pixel 7", brand: "Google", stock: 20 },
  { model: "Nord N20", brand: "OnePlus", stock: 15 },
  { model: "Xperia 1", brand: "Sony", stock: 5 }
])
```

Instructor Scenario: Multi-Brand Filtering

Scenario: A customer uses the sidebar checkboxes to filter smartphones. They only want to see phones made by "Apple", "Samsung", or "Google".

The Query:

```
JavaScript
db.phones.find({
  brand: { $in: ["Apple", "Samsung", "Google"] }
})
```

Student Exercise

☐ **Scenario:** You want to run a clearance sale on lesser-known brands. Write a query to find all phones that are NOT made by "Apple" or "Samsung".

Topic 3 - Logical Operators

Explanation of Elements

These are used for combining different fields to create complex rules:

\$or

Returns a document if it matches at least one of the conditions provided in an array.

\$and

Returns a document only if it matches all the conditions provided.

Setup Data (Run this first!)

```
JavaScript
db.inventory.insertMany([
  { name: "Winter Coat", section: "Clearance", discountPercentage: 60, price: 40 },
  { name: "Summer T-Shirt", section: "Standard", discountPercentage: 10, price: 20 },
  { name: "Running Shoes", section: "Standard", discountPercentage: 55, price: 60 },
  { name: "Scarf", section: "Clearance", discountPercentage: 20, price: 15 }
])
```

Instructor Scenario: The Flash Sale

Scenario: You want to highlight special deals on the homepage. An item qualifies as a deal if it is in the "Clearance" section OR if its discount percentage is greater than 50%.

The Query:

```
JavaScript
db.inventory.find({
  $or: [
    { section: "Clearance" },
    { discountPercentage: { $gt: 50 } }
  ]
})
```

Student Exercise

- ❑ **Scenario:** A customer has a "\$50 Bill" in their pocket. Write a query using \$and to find items that are in the "Standard" section AND cost less than or equal to 50. (Hint: While MongoDB does implicit \$and, write it out explicitly for practice!)

Topic 4 - Cursor Methods (Formatting Output)

Explanation of Elements

Once MongoDB finds the documents, how do we format the output?



.sort()

Orders your results (1 for Ascending, -1 for Descending).



.limit()

Restricts the maximum number of documents returned.



.skip()

Skips a specified number of documents before returning the rest.

 **Note:** Combining `.limit()` and `.skip()` is exactly how frontend pagination works!

Setup Data (Run this first!)

```
JavaScript
db.highscores.insertMany([
  { player: "Alice", score: 1500 },
  { player: "Bob", score: 3200 },
  { player: "Charlie", score: 900 },
  { player: "Dave", score: 4100 },
  { player: "Eve", score: 2750 },
  { player: "Frank", score: 5000 }
])
```

Instructor Scenario: The Top 3 Leaderboard

Scenario: You are building the leaderboard screen for a video game. You need to grab the top 3 highest scores in the database to display on the podium.

The Query:

```
JavaScript
db.highscores.find().sort({ score: -1 }).limit(3)
```

Student Exercise

 **Scenario:** Your leaderboard frontend shows 2 players per "page". Write the query to fetch the data for Page 2. (You need to sort them highest to lowest, skip the players on Page 1, and limit the result to 2 players).